Introduction

REST

REST

- Representational State Transfer
- In 2000 by Roy Fielding
- Client-server architecture
- WWW implements REST

The client-server model defined by the WWW is theorized by Roy Fielding in his PhD theses in 2000. The architectural style of WWW can be reused in other applications, most importantly in web services (called RESTful WebServices). One of the designers of the HTTP was Fielding.

RESTful Communication

- Resources are represented in documents
  - Capturing the state of the resource
- Requests and responses
  - Transfers of representations of resources
- A client is ready to make a state transition
  - Initiates a request
- There are one or more outstanding requests
  - The client is in transition
- The resource contains the links to the next state transitions

In a RESTful architecture, a resource is identified by an URI. Using the URL the client can obtain some representation of the resource. For example, in WWW, the resource is a text, which is represented in an HTML document. The document contains links to available states, the client can choose, and can initiate a request.

Design goals

- Scalability of component interactions
- Generality of interfaces
- Independent deployment of components
- Intermediary components to reduce latency, enforce security and encapsulate legacy systems

Design constraints

- Client-server
- Stateless
- Cacheable
- Uniform interface
- Layered system
- Code-on-demand

The design space of REST was to design a client-server architecture. To improve scalability and reliability, the server should be stateless. There is no state to store on server-side and share between servers, the applications are more scalable. Responses from the server are reusable, reducing network traffic. Uniform interface decouples implementations and services.

Layering provides information hiding, an inner layer cannot be visible from the outside. For example, the client cannot tell that the server it connected to is the real server holding the resource, or an intermediate server (such as load balancer or caching proxy).

Code-on-demand is an optional feature. With code on demand, the server can extend the functionality of the client (e.g. JavaScript in WWW).

RESTful API design guidelines

- Identification of resources
- Manipulation of resources through representations
- Self-descriptive messages
- Hypermedia as the engine of application state

In a REST system, each request manipulates a resource. A REST system should have a language to describe the resource. Such a language is URI. The representation is conceptually different from the resource itself. Resource is a database record, but it can be represented e.g. in XML, JSON or HTML. These resources are manipulated by their representations. There should be enough meta-data attached to the representation (such as internet media type) to be self-descriptive. The client should not assume anything about the server, just the initial resource. Every possible state transition must be identified by the representation of the resource dynamically.

RESTful WebService characteristics

- Independent of protocols (not just HTTP)
- Should not change the communication protocoll

    - Unlike WebDav or SVN
- Should be described by the media-type
    - Without extra information
- Should not have any fixed resource name or hierarchy
    - Except the initial URI
- Strictly not typed
    - Except the media-type
- REST client should not have more knowledge than the initial URI
    - The client chooses the next state from the representation
    - Based on the client's capabilities

JAX-RS

JAX-RS API

- JavaSE 5, JavaEE 6
    - JSR311: JAX-RS: The Java API for RESTful Web Services
- JavaEE 7
    - JSR339: JAX-RS 2.0: The Java API for RESTful Web Services
    - Reference implementation: Jersey

JAX-RS API provides a plenty of annotations and interfaces for developers. Annotations in general are used on the server side to define which service should serve a given request URI, and which method should be called for the corresponding HTTP method and accepted media types.

Hello World

In this section, we create a simple RESTful web service.

Server side

Configuring the servlet container

```java
import javax.ws.rs.ApplicationPath;
import javax.ws.rs.core.Application;

@ApplicationPath("rest")
public class RESTConfig extends Application {
}
```

Our services will be executed by the Jersey engine. Jersey is the reference implementation of the JAX-RS 2.0 specification. The engine itself runs inside a servlet container, so it is needed to be configured. This can be done in several ways. The example shows the easiest method for servlet containers supporting servlet specification 3.0. By extending `javax.ws.rs.core.Application`, and annotating it with `@ApplicationPath`. Its parameter sets where Jersey search for webservices. Jersey engine servlet will be configured to an URL pattern based on this parameter (in this case `/rest/*`, which is relative to the application's context path).

Hello World service

The service will be accessible at:`http://<host>:<port>/<context-path>/rest/hello`

```java
@Path("/hello")
public class Hello {

    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String sayPlainTextHello() {
        return "Hello Jersey";
    }

    ...
```

In the class above, a simple service is implemented. The URL of the service is `/hello`, relative to the URL of the Jersey servlet. So, this webservice can be accessed by the URL `/<context-path>/rest/hello`, where `/rest` is the root URL of the Jersey servlet itself.

The defined method responses for GET queries, and media type of the response is `text/plain`. The same resouce can have multiple representations. For instance, to say hello world in XML, we can extend the class with a new method that can be seen on the next slide.

In the case of HTTP, the client and server can use content-negotiation based on the Accept request header.

Hello World service (XML)

```
    ...

    @GET
    @Produces(MediaType.TEXT_XML)
    public String sayXMLHello() {
        return "<?xml version=\"1.0\"?>"
             + "<greeting>Hello RESTful World!</greeting>";
    }

    ...
```

The name of the service method is irrelevant. Just the request method annotation and the internet media type of the returned representation matters.

Client side

Client

```
public class Client1 {
  public static void main(String[] args) {
    Client client = ClientBuilder.newClient();
    WebTarget target = client.target("http://localhost:8080/hello");
    WebTarget target2 = target.path("rest").path("hello");
    Invocation.Builder invocation =
      target2.request(MediaType.TEXT_PLAIN);
    System.out.println(invocation.get(String.class));
    // Alternatively, the same call chain:
    System.out.println(
      client.target("http://localhost:8080/hello")
        .path("rest/hello")
        .request(MediaType.TEXT_XML)
        .get(String.class)
    );
  }
}
```

With the help of `ClientBuilder`, we can acquire an implementation-specific instance from the `Client` class. It enables us to specify a target `URI` in one or more steps. By specifying the accepted internet media type, we get an invocation builder, which can be used to add other headers to the request. The request itself can be sent with a method which follows the corresponding `HTTP` method name, in this case `get(...)` will send a `GET` request. This call will return with the extracted entity from the message body, in this case typed as a `java.lang.String`.

Note that we assumed `hello` as context-path of the application, and the server runs on `localhost`, using port 8080.

Client

```
public class Client2 {
  public static void main(String[] args) {
    Client client = ClientBuilder.newClient();

    WebTarget target =
      client.target("http://localhost:8080/hello/rest/hello");

    Response response =
      target.request(MediaType.TEXT_PLAIN_TYPE).get();

    System.out.println(response.getStatus());
    if (response.hasEntity()) {
      System.out.println(response.readEntity(String.class));
    }
  }
}
```

If we want to access other parts of a response than the entity in the message body, we can use an argumentless version of each sender method. In this case it returns a `Response` object which provides various getter methods to extract different parts of the response message, including headers, response status code and message body.

`API` details

Methods and entities

Methods

HEAD Same as GET, but the response does not contain the resource, only the headers.

GET Query a resource. Idempotent and safe.

PUT Create a resource. Idempotent.

DELETE Delete a resource. Idempotent.

POST Create or modify a resource.

`HTTP 1.0` offers more methods, but these can be used in RESTful webservices. Recall that safety means the method does not change any state on the server, and issuing the same request with an idempotent method twice is the same as sending it only once. Each method have a corresponding annotation in the `javax.ws.rs` package.

Entitites (client side)

```
Form form = new Form();
form.param("id", "42");

String result =
    client.target("http://localhost:8080/hello")
          .path("rest/hello")
          .request()
          .post(Entity.form(form), String.class);
System.out.println(result);
```

Using request entities we can send data in the message body. The methods `POST`, `PUT` and `DELETE` can have both request and response entities. Handling of such a response entity is shown on the above example, where the simplest type of entity (namely a form) is passed to the server side.

Entities (server side)

```
@POST
@Consumes(MediaType.APPLICATION_FORM_URLENCODED)
public Response form(@FormParam("id") String id) {
  if ("42".equals(id)) {
    return Response.ok("Done", MediaType.TEXT_PLAIN).build();
  } else {
    return Response.status(Response.Status.BAD_REQUEST).build();
  }
}
```

On the server side, form fields can be received even as separate parameters. This is achieved by annotating arguments with `@FormParam` and specifying the desired field name. Of course, every field data will arrive as a string. The example also shows how to use the builder facilities of `Response` class to specify status code and response entity.

Request parameters

More complex cases can also be handled, where path and query parameters are used. Both of them are transferred as a part of the request `URI`.

Path parameter mapping

```
@Path("/users/{department}")
class UserService {
  @GET
  @Path("/{identifier}")
  @Produces(MediaType.TEXT_PLAIN)
  public String list(@PathParam("department") String department,
                     @PathParam("identifier") String identifier) {
    ..
  }
}
```

Accessible as e.g. `.../users/sales/1337-42`

By introducing { and } as markers in the paths, we can extract various parts as named variables later. If the requested `URI` cannot be mapped to any method, Jersey will return a 404 Not Found response. `@Path` annotation, when used with methods, introduces a level of sub-paths. Thus the example method above can be addressed as e.g. `.../users/sales/1337-42`, and it will be called with department ``sales'' and identifier ``1337-42''.

Query parameter mapping

```
@Path("/users")
class UserService {
  @GET
  @Produces(MediaType.TEXT_PLAIN)
  public String list(@QueryParam("department") String department,
                     @QueryParam("identifier") String identifier) {
    ..
  }
}
```

Accessible as e.g. `.../users?identifier=1337-42&department=sales`

Parameters can also be extracted from query strings. A query string contains key-value pairs in form `key=value` separated by ampersands (`&`). Unlike in the case of path parameters, the order of query string arguments is irrelevant.

Header parameter mapping

```
@Path("/users")
class UserService {
  @GET
  @Produces(MediaType.TEXT_PLAIN)
```

```
  public String list(@HeaderParam("X-Department") String department) {
    ..
  }
}
```

The request must contain an `X-Department` header

Using `@HeaderParam` annotation, any request header can be matched by name and injected as a parameter to the service method.

Of course, different kind of parameters can be used in the same method, so it is possible to have header, query, path, and form parameters in the same argument list. For extraction of cookie values, use `@CookieParam`.

Parameter injection

```
@Path("/db/{id}")
public class DB {
    private @PathParam("id") @DefaultValue("0") String id;
    private @Context UriInfo uriInfo;

    @HeaderParam("X-Test")
    private void setTest(String test) {
        ...
    }
}
```

`@Context`

- Application
- UriInfo
- Request
- HttpHeaders
- SecurityContext
- Providers

Parameters can be injected also in class fields, not only as method arguments. The same annotations can be used to inject headers, path and query string parameters, form fields and cookies. `@Context` provides injection of additional context information objects. If some parameters are optional, we can define default values for them using `@DefaultValue`.

Default values

- What can be annotated with `@DefaultValue`?

  - Primitive type
  - Have a constructor with String argument
  - Have a static method `valueOf` or `fromString`
  - `List<T>`, `Set<T>` or `SortedSet<T>`, where `T` satisfies the above requirements
- These requirements are generally true for all argument types used with request parameter extractor annotations

Lifecycle

Lifecycle of a service

- Request scope is the default
- Per-lookup scope: an instance is created for every lookup
- Singleton if annotated by `javax.inject.Singleton`
- Other `CDI` and `EJB` annotations
  - When applicable
  - E.g. cannot be `@Stateful`

By default, a new instance of the service class is created to serve every request. However, we can control this behavior with some of the `CDI` and `EJB` annotations. For example, marking a service class with `@Singleton` will ensure that only one instance will be created by the container in the same virtual machine.

Lifecycle and injection

```
@Singleton
@Path("/db/{id}")
public class DB {
    @PathParam("id")
    private String id;

    @Context
    private UriInfo uriInfo;

    ...
}
```

- `id` is not proxiable, thus it does not work
- `UriInfo` works, however

Sometimes the lifecycle of the service and its field injections can interfere. For example, various parameter annotations, like `@PathParam` are associated with single request. When lifetime of the service itself is longer than a request, it is impossible to solve this kind of injections.

Building response

Building response

```
@GET
@Path("/images/{image}")
@Produces("image/*")
public Response getImage(@PathParam("image") String image) {
  if (!isSafeToOpenFile(image)) {
    throw new IllegalArgumentException("Cannot open the file.");
  }

  File f = new File(image);
  if (!f.exists()) {
    throw new WebApplicationException(404);
  }

  String mt = new MimetypesFileTypeMap().getContentType(f);
  return Response.ok(f, mt).build();
}
```

Builder methods of `Response` class are useful to easily create responses from various types and entity sources. The example above shows that even a file object is directly usable for this task. Of course, the service will send the file contents in the response body. The exact internet media type is determined by a utility class, which analyzes the file's extension.

Response types

- `Response` class factory methods
  - Different methods for frequent status codes
  - `ok()`, `created()`, `temporaryRedirect()`, `serverError()`, etc.
  - The first can insert event a repsonse entity in one step
  - A general method for any status code: `status(code)`
  - Every factory method returns a `ResponseBuilder`
- `ResponseBuilder`
  - `entity()` inserts a given response entity

- `header()` adds a header to the response
- Other methods for different headers like `Content-Type`, etc.
- `build()` creates a response object which can be returned

```
Response.ok(xmlData, MediaType.TEXT_XML)
        .header('X-Custom-Header', 'my value')
        .build();
```

XML binding

Java Architecture for XML Binding (JAX-B) specification defines an API to serialize Java objects into XML. The serialization process is controlled by annotations to determine the XML representation of each field of a compound object.

Tree example

```
@Singleton
@Path("/tree")
public class Tree {
  private Map<String, TreeNode> db = new ConcurrentHashMap<>();

  public Tree() {
    TreeNode root = new TreeNode("0", "ROOT");
    db.put(root.getKey(), root);
    root.getChildren().add(new TreeNode("1", "A"));
    root.getChildren().add(new TreeNode("2", "B"));
    for (TreeNode node : root.getChildren()) {
      db.put(node.getKey(), node);
    }
    System.out.println("RELOAD");
  }
```

Tree example

```
  @GET
  public Response get() {
    return getId("0");
  }

  @GET @Path("/{id}")
  @Produces(MediaType.APPLICATION_XML)
  public Response getId(@PathParam("id") String id) {
    TreeNode node = db.get(id);
```

```
    if (node == null) {
      return Response.status(404).build();
    } else {
      return Response.ok(node).build();
    }
  }
}
```

Using internet media type `application/xml` automatically enforces `XML` serialization of the data with `JAXB`.

Client

```
public class TreeClient {
  public static void main(String[] args) {
    Client client = ClientBuilder.newClient();
    String baseURI = "http://localhost:8080/hello";
    System.out.println(client.target(baseURI)
                             .path("rest/tree").request()
                             .get(String.class));
    Response response = client.target(baseURI)
                              .path("rest/tree/1").request()
                              .get();
    if (response.getStatus() == 200) {
      System.out.println(response.readEntity(TreeNode.class));
    }
  }
}
```

If the type of the transferred entity (in this case `TreeNode`) is known both in client and server, it can be easily deserailized from `XML` on the client side.

Node entity with `JAXB`

```
@XmlRootElement
@XmlAccessorType(XmlAccessType.NONE)
public class TreeNode {
  private String key;
  private String value;
  private List<TreeNode> children = new ArrayList<>();

  public TreeNode() {}
```

```
  public TreeNode(String key, String value) {
    this.key = key;
    this.value = value;
  }
```

Node entity with `JAXB`

```
  @XmlAttribute @XmlID
  public String getKey() { return key; }

  @XmlElement
  public String getValue() { return value; }

  @XmlElementWrapper
  @XmlElement(name="treeNode")
  @XmlIDREF
  public List<TreeNode> getChildren() {
    return children;
  }
}
```

The next examples will show the effect of the different `JAXB` annotations on method `getChildren()`.

Using `XmlElement`

```
@XmlElement
public List<TreeNode> getChildren() {
    return children;
}
```

Result

```
<treeNode key="0">
    <children key="1"><value>A</value></children>
    <children key="2"><value>B</value></children>
    <value>ROOT</value>
</treeNode>
```

Using `XmlElementRef`

```
@XmlElementRef
public List<TreeNode> getChildren() {
    return children;
}
```

Result

```
<treeNode key="0">
    <treeNode key="1"><value>A</value></treeNode>
    <treeNode key="2"><value>B</value></treeNode>
    <value>ROOT</value>
</treeNode>
```

Using `XmlIDREF`

```
@XmlIDREF
public List<TreeNode> getChildren() {
    return children;
}
```

Result

```
<treeNode key="0">
    <children>1</children>
    <children>2</children>
    <value>ROOT</value>
</treeNode>
```

Using `XmlElementWrapper`

```
@XmlElementWrapper @XmlElement(name="treeNode") @XmlIDREF
public List<TreeNode> getChildren() {
    return children;
}
```

Result

```
<treeNode key="0">
    <children>
        <treeNode>1</treeNode>
        <treeNode>2</treeNode>
    </children>
    <value>ROOT</value>
</treeNode>
```

JAX-WS

Java API for XML Web Services (JAX-WS) is a technology which enables Java developers to easily create SOAP and WSDL-based web services. It provides tools (wsimport and wsgen) to generate the necessary artifacts to use and provide a web service.

Service

```
@WebService
@Stateless
public class TemperatureConverter {
    public double celsiusToFahrenheit(double c) {
        return 9.0 / 5.0 * (c + 32.0);
    }
}
```

This example uses the javax.jws.WebService annotation to mark a stateless EJB as a JAX-WS web service.

Client

```
public class Main {
  public static void main(String[] args)
      throws ServiceException, RemoteException {
    TemperatureConverter converter =
      new TemperatureConverterService()
                .getTemperatureConverterPort();
    System.out.println(converter.celsiusToFahrenheit(42));
  }
}
```

The TemperatureConverterService class is generated from the WSDL description of the service with wsimport.